```
----Mounting QEMU Image----
add map loop0p1 (254:0): 0 2095104 linear 7:0 2048
----Creating Filesystem----
mke2fs 1.43.9 (8-Feb-2018)
Discarding device blocks: done
Creating filesystem with 261888 4k blocks and 65536 inodes
Filesystem UUID: 5a3d258b-879e-4b5f-889f-bd4a1bebed90
Superblock backups stored on blocks:
        32768, 98304, 163840, 229376

Allocating group tables: done
Writing inode tables: done
Writing superblocks and filesystem accounting information:
done

----Making QEMU Image Mountpoint----
----Mounting QEMU Image Partition 1----
----Extracting Filesystem Tarball----
----Creating FIRMADYNE Directories----
----Patching Filesystem (chroot)----
Creating /etc/TZ!
Creating /etc/hosts!
Creating /etc/passwd!
Removing /etc/scripts/sys_resetbutton!
----Setting up FIRMADYNE----
----Unmounting QEMU Image----
```

Having created an image that can be emulated, the network interface can be kick-started and the firmware can be emulated, as shown in the code below.

```
root@kali:~/firmadyne# ./scripts/inferNetwork.sh 8
Querying database for architecture... Password for user
firmadyne:
mipseb
Running firmware 8: terminating after 60 secs...
qemu-system-mips: -net nic,vlan=0: 'vlan' is deprecated.
Please use 'netdev' instead.
Bad SWSTYLE=0x04
qemu-system-mips: terminating on signal 2 from pid 2074
(timeout)
Inferring network...
Interfaces: [('br0', '192.168.0.1'), ('br1', '192.168.7.1')]
Done!
```

Now, in this case, the interface *eth0* of the DLink firmware is up with the IP *192.168.0.1* and *192.168.7.1*. Once the interface is up, the firmware can be emulated with the *run.sh* script, as shown in Figure 1.

Since the DLink firmware used in this demonstration consists of a Web portal, it can be accessed over *http://192.168.0.1* from any Web browser, as shown in Figure 2.

Next, after mounting the firmware, tests can be carried out for various application level security bugs. Tools like Nmap can be used to further probe for open ports. This enables an understanding of other possible entry points into the application. In a similar way, dynamic assessment can be carried out for different types of firmware.

```
root@kali:~/firmadyne# ./scratch/5/run.sh
Creating TAP device tap5_0...
Set 'tap5_0' persistent and owned by uid 0
Bringing up TAP device...
Adding route to 192.168.0.1...
Starting firmware emulation... use Ctrl-a + x to exit
qemu-system-mips: -net nic,vlan=0: 'vlan' is deprecated. Please use 'netdev' instead.
[    0.000000] Linux version 2.6.32.70 (vagrant@vagrant-ubuntu-trusty-64) (gcc version
[    0.000000]
[    0.000000] LINUX started...
[    0.000000] bootconsole [early0] enabled
[    0.000000] CPU revision is: 00019300 (MIPS 24Kc)
[    0.000000] FPU revision is: 00739300
[    0.000000] Determined physical RAM map:
[    0.000000]  memory: 00001000 @ 00000000 (reserved)
[    0.000000]  memory: 000ef000 @ 00001000 (ROM data)
[    0.000000]  memory: 0061e000 @ 000f0000 (reserved)
[    0.000000]  memory: 0f8f1000 @ 0070e000 (usable)
[    0.000000] debug: ignoring loglevel setting.
[    0.000000] Wasting 57792 bytes for tracking 1806 unused pages
[    0.000000] Initrd not found or empty - disabling initrd
[    0.000000] Zone PFN ranges:
[    0.000000]   DMA      0x00000000 -> 0x00001000
[    0.000000]   Normal   0x00001000 -> 0x0000ffff
[    0.000000] Movable zone start PFN for each node
```

Figure 1: *run.sh* script will emulate the firmware as per the ID assigned by the *extractor.py* script

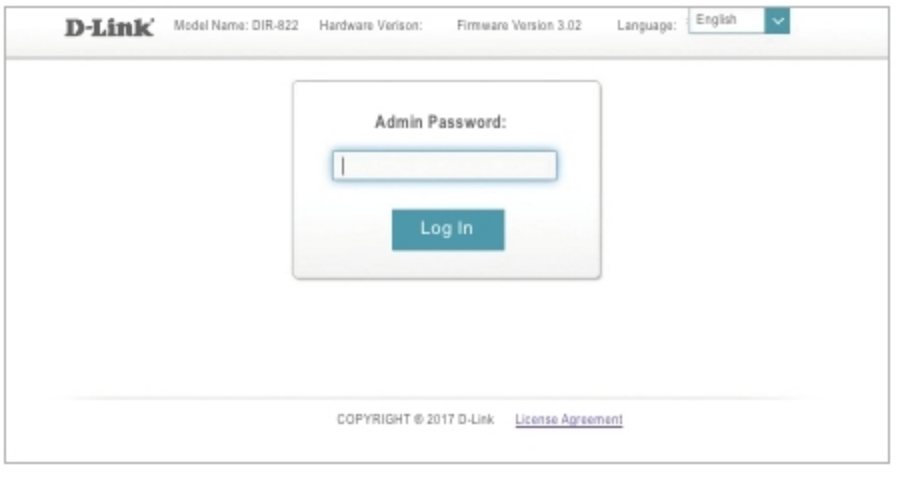

Figure 2: Post emulation when you access the Web console of given firmware

Firmadyne comes with three basic automated tools — accessible Web page, analyse snmpwalk and vulnerability check. Accessible Web page script goes through the file system of the firmware image and gathers the information depending on the authentication required. Analyse snmpwalk gathers information about the SNMP (simple network management protocol) community strings. The vulnerability check module tests for the presence of vulnerabilities using exploits from Metasploit. Using interceptor tools like burpsuite, we can find the Web based vulnerabilities after emulating the firmware. This can be done without having access to expensive hardware

So with the help of Firmadyne we can automate firmware analysis to an extent. However, tester intervention is always required to verify the bugs. Also, emulating the images becomes easy with Firmadyne. END

References

[1] *https://github.com/firmadyne*
[2] *https://www.qemu.org*

By: Suhas Nayak

The author works as a security researcher at Reliance Jio. He can be reached at *suhas.in@live.com*.